

SV9 - Digital Verification using SV & UVM

===== Project =====

Synchronous FIFO verification using UVM

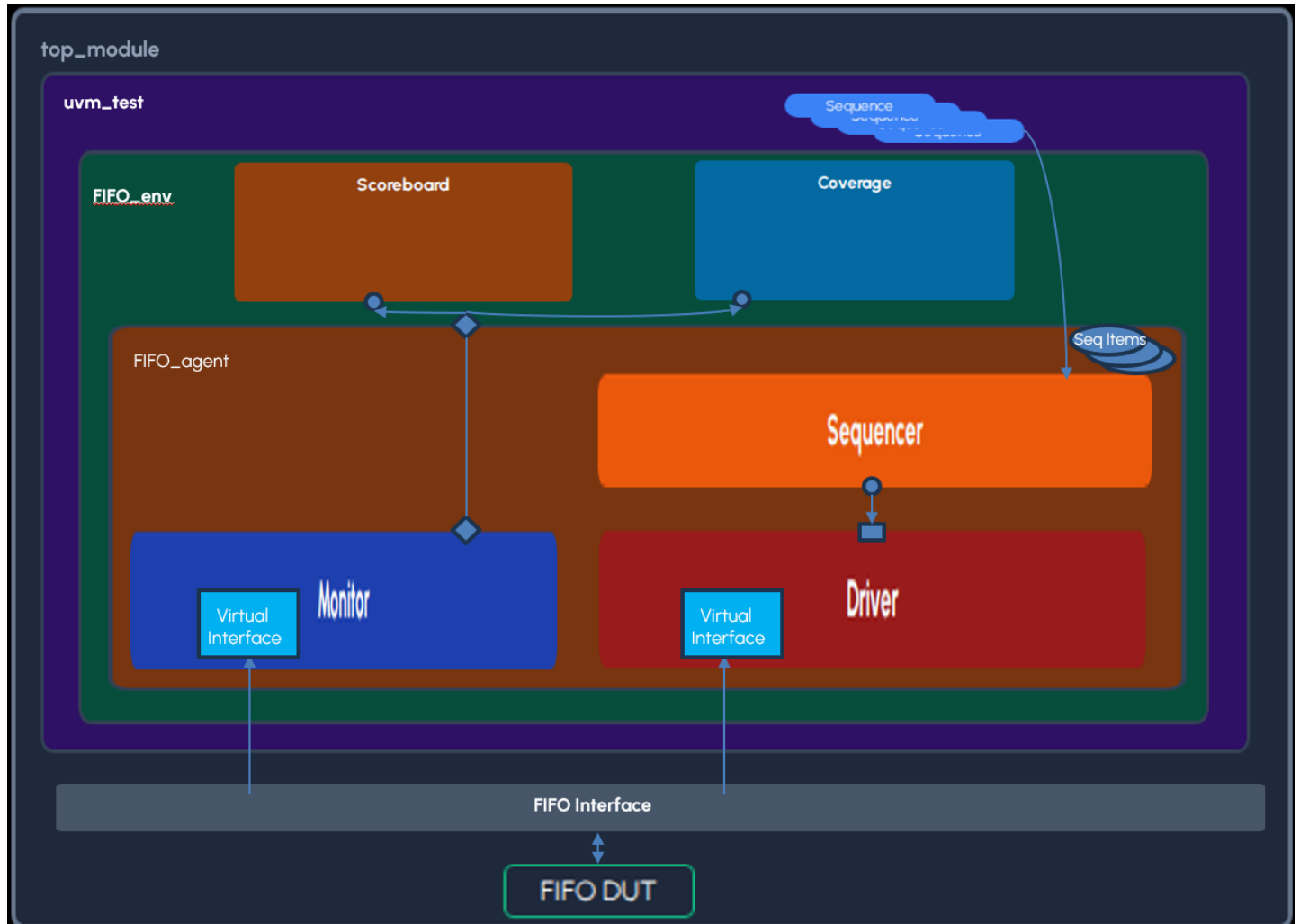
Name

Ibrahim Mohamed Mohamed Hanafy

-Verification Plan

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
Reset	When the rst_n is asserted, internal counters and pointers are reset to 0, leading to an empty FIFO	Directed at the start of the simulation, then randomized such that it is off most of the time	-	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_count -> assertion check: a_pointers
Writing 1	When the wr_en is asserted, the data_in is written to an empty entry, if available.	Randomization, ST: It is active for a user set amount of time	Cover all cases with rd_en and each o/p control signal	A check_data function in a scoreboard class, that checks against a reference_model.
Writing 2	If writing was successful, wr_ack conveys acknowledgment of the writing process, and vice versa	"" ""	"" ""	-> assertion check: a_wr_ack_1 (ack) -> assertion check: a_wr_ack_2 (no ack)
Reading	When the rd_en is asserted, the data_out is read from an occupant entry, if available. The data read's entry is now free for another	Randomization, ST: It is active for a user set amount of time	Cover all cases with wr_en and each o/p control signal	A check_data function in a scoreboard class, that checks against a reference_model.
Writing and Reading	Reading has priority for full FIFO Writing has priority for an empty FIFO Simultaneous for a normal FIFO	Randomization, ST: Both are active for a user set amount of time	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model.
Overflowing	If the FIFO is full, and wr_en was asserted, the writing process is ignored, and the elements don't change, and a flag is raised	Randomization, ST: Writing is more probable than reading	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_overflow
Underflowing	If the FIFO is empty, and rd_en was asserted, the reading process is ignored, and the elements don't change, and a flag is raised	Randomization, ST: Reading is more probable than reading	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_underflow
Full	If the wr_en is asserted, and there all entries have been written to and not read yet, a full flag is raised	Randomization, ST: Writing is more probable than reading	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_full
Empty	If the rd_en is asserted, and there is no entries that were written to, an empty flag is raised	Randomization, ST: Reading is more probable than reading	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_empty
Almost Full	If there is only one entry remaining for writing, the almost full flag is raised	Randomization, ST: Writing is more probable than reading	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_almost_full
Almost Empty	If there is only one entry remaining for writing, the almost empty flag is raised	Randomization, ST: Reading is more probable than reading	Cover all cases with wr_en and rd_en	A check_data function in a scoreboard class, that checks against a reference_model. -> assertion check: a_almost_empty
Read Pointer Threshold	The Read pointer threshold value = depth of FIFO (cant ever surpass it)	Randomization	-	-> assertion check: a_rd_ptr_threshold
Read Pointer Threshold	The Write pointer threshold value = depth of FIFO (cant ever surpass it)	Randomization	-	-> assertion check: a_wr_ptr_threshold
Read Pointer wrapping	The read pointer wraps around after reaching its threshold value	Randomization	-	-> assertion check: a_rd_ptr_wrap
Write Pointer wrapping	The write pointer wraps around after reaching its threshold value	Randomization	-	-> assertion check: a_wr_ptr_wrap

-UVM Testbench Diagram



Results

-Code Coverage

===== === File: ../../VS_code/Project_FIFO/Enviroment/FIFO.sv =====				
Statement Coverage:				
Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	----	-----	-----
Stmts	22	22	0	100.0

Branch Coverage:				
Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	----	-----	-----
Branches	25	25	0	100.0

Toggle Coverage:				
Enabled Coverage	Active	Hits	Misses	% Covered
-----	-----	----	-----	-----
Toggle Bins	20	20	0	100.0

-Functional Coverage

TYPE	CovPort	FIFO_cover...	100.0%	100	100.0%		✓
+ CVP	CovPo...	FIFO_cover...	100.0%	100	100.0%		✓
+ CVP	CovPo...	FIFO_cover...	100.0%	100	100.0%		✓
+ CVP	CovPo...	FIFO_cover...	100.0%	100	100.0%		✓
+ CVP	CovPo...	FIFO_cover...	100.0%	100	100.0%		✓
+ CVP	CovPo...	FIFO_cover...	100.0%	100	100.0%		✓
+ CVP	CovPo...	FIFO_cover...	100.0%	100	100.0%		✓
+ CVP	CovPo...	FIFO_cover...	100.0%	100	100.0%		✓
+ CVP	CovPo...	FIFO_cover...	100.0%	100	100.0%		✓
+ CVP	CovPo...	FIFO_cover...	100.0%	100	100.0%		✓
+ CROSS	Co...	FIFO_cover...	100.0%	100	100.0%		✓
+ CROSS	Co...	FIFO_cover...	100.0%	100	100.0%		✓
+ CROSS	Co...	FIFO_cover...	100.0%	100	100.0%		✓
+ CROSS	Co...	FIFO_cover...	100.0%	100	100.0%		✓
+ CROSS	Co...	FIFO_cover...	100.0%	100	100.0%		✓
+ CROSS	Co...	FIFO_cover...	100.0%	100	100.0%		✓
+ CROSS	Co...	FIFO_cover...	100.0%	100	100.0%		✓

-Assertions Coverage

▲ /top/dut/a_wr_ack_1_cover	SVA	✓	Off	823	1	Unli...	1	100%		✓
▲ /top/dut/a_wr_ack_2_cover	SVA	✓	Off	88	1	Unli...	1	100%		✓
▲ /top/dut/a_rd_ptr_wrap_cov...	SVA	✓	Off	22	1	Unli...	1	100%		✓
▲ /top/dut/a_wr_ptr_wrap_co...	SVA	✓	Off	46	1	Unli...	1	100%		✓
▲ /top/dut/a_wr_ptr_threshold...	SVA	✓	Off	2813	1	Unli...	1	100%		✓
▲ /top/dut/a_rd_ptr_threshold...	SVA	✓	Off	2813	1	Unli...	1	100%		✓
▲ /top/dut/a_count_range_cov...	SVA	✓	Off	2813	1	Unli...	1	100%		✓
▲ /top/dut/a_full_cover	SVA	✓	Off	132	1	Unli...	1	100%		✓
▲ /top/dut/a_empty_cover	SVA	✓	Off	1571	1	Unli...	1	100%		✓
▲ /top/dut/a_almost_full_cover...	SVA	✓	Off	112	1	Unli...	1	100%		✓
▲ /top/dut/a_almost_empty_co...	SVA	✓	Off	433	1	Unli...	1	100%		✓
▲ /top/dut/sva_instance/a_ov...	SVA	✓	Off	58	1	Unli...	1	100%		✓
▲ /top/dut/sva_instance/a_un...	SVA	✓	Off	803	1	Unli...	1	100%		✓

-Bug Report

Bug #1: Underflow signal not implanted sequentially

Bug #2: almost full implemented incorrectly

**Bug #3: incorrect resetting behavior:
wr_ack, underflow, overflow, data_out**

Bug #4: No handling of simultaneous read and writing

Bug #5: Incorrect conditions for overflow and underflow

-Sequences and Waveforms

*Reset sequence

```
//Reset sequence=====
class FIFO_reset_seq extends uvm_sequence #(FIFO_seq_item)
  `uvm_object_utils(FIFO_reset_seq)

  FIFO_seq_item seq_item;

  function new(string name = "FIFO_reset_seq");
    super.new(name);
  endfunction

  task body;
    seq_item = FIFO_seq_item::type_id::create("seq_item");

    start_item(seq_item);
    seq_item.rst_n = 0;
    seq_item.wr_en = 0;
    seq_item.rd_en = 0;
    seq_item.data_in = '0;
    finish_item(seq_item);
  endtask
endclass
```

Signal	Value	Signal	Value
clk	1'h0		
data_in	16'h57aa	0000	cb8a
rst_n	1'h1		d1ac
wr_en	1'h1		
rd_en	1'h0		
data_out	16'h0000	0000	
wr_ack	1'h1		
overflow	1'h0		
full	1'h0		
empty	1'h0		
almostfull	1'h0		
almostempty	1'h0		
underflow	1'h0		
mem	16'hxxxx 16'hxxx...	xxxx xxxx xxxx xxxx xxxx xx...	xxxx xxxx xxxx xx
[7]	16'hxxxx		
[6]	16'hxxxx		
[5]	16'hxxxx		
[4]	16'hxxxx		
[3]	16'hxxxx		
[2]	16'he475		
[1]	16'hd1ac		
[0]	16'hcb8a		cb8a

*Write only sequence

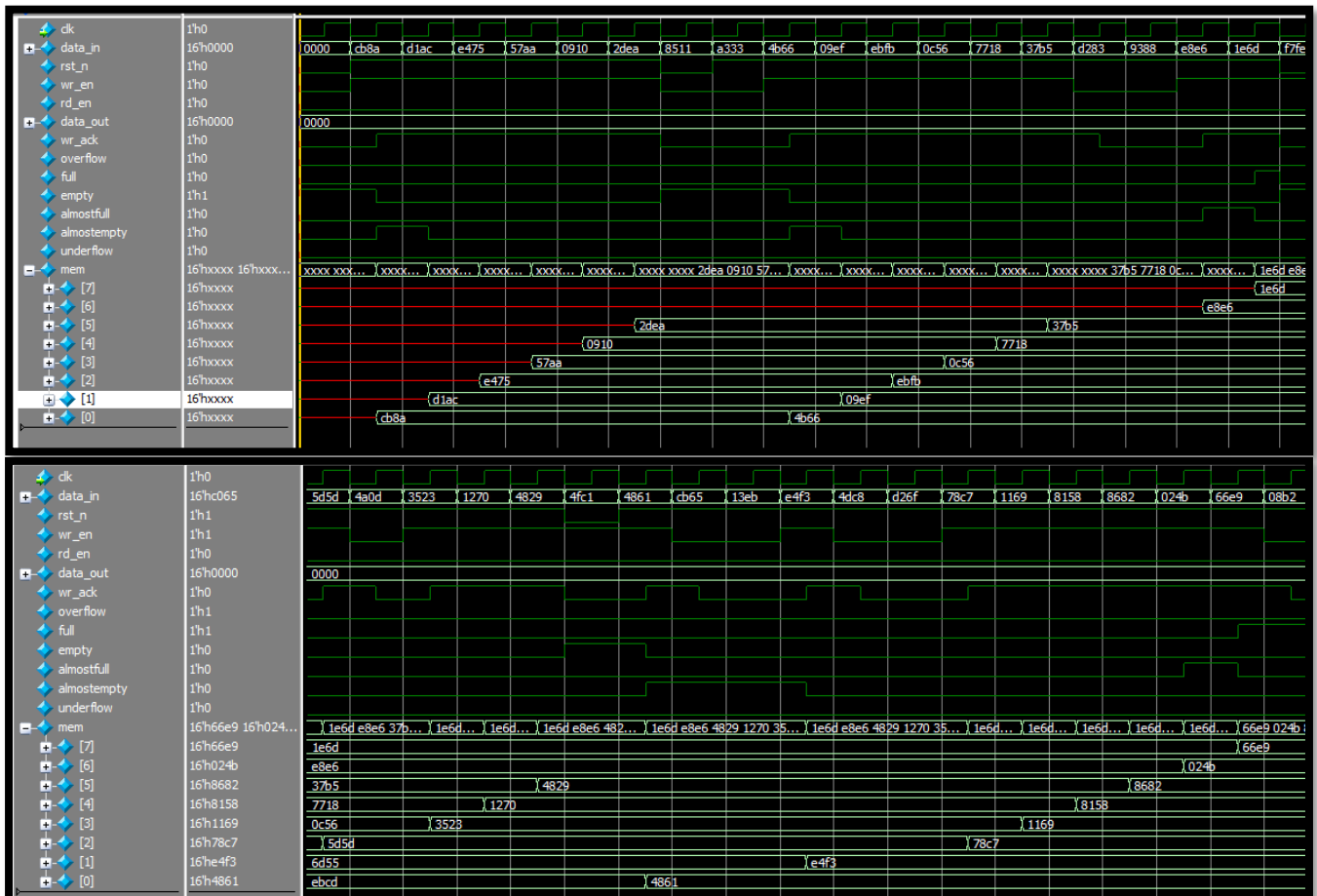
```
class FIFO_wr_only_seq extends uvm_sequence #(FIFO_seq_item);
  `uvm_object_utils(FIFO_wr_only_seq)

  FIFO_seq_item seq_item;

  function new(string name = "FIFO_wr_only_seq");
    super.new(name);
  endfunction

  task body;
    seq_item = FIFO_seq_item::type_id::create("seq_item");

    repeat(100) begin
      start_item(seq_item);
      seq_item.RD_EN_ON_DIST = 0;
      seq_item.WR_EN_ON_DIST = 70;
      assert(seq_item.randomize());
      finish_item(seq_item);
    end
  endtask
endclass
```



*Write only sequence

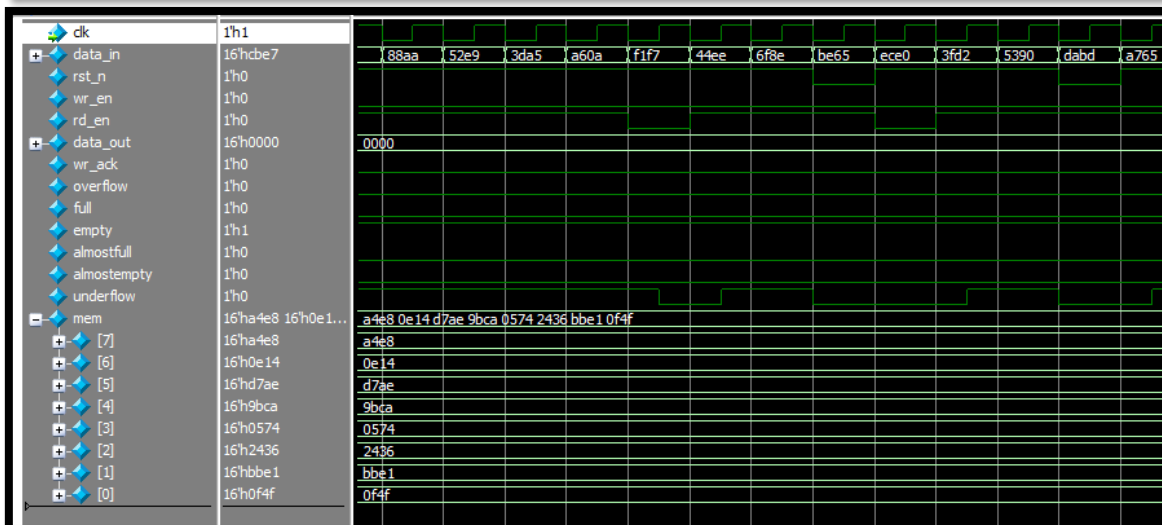
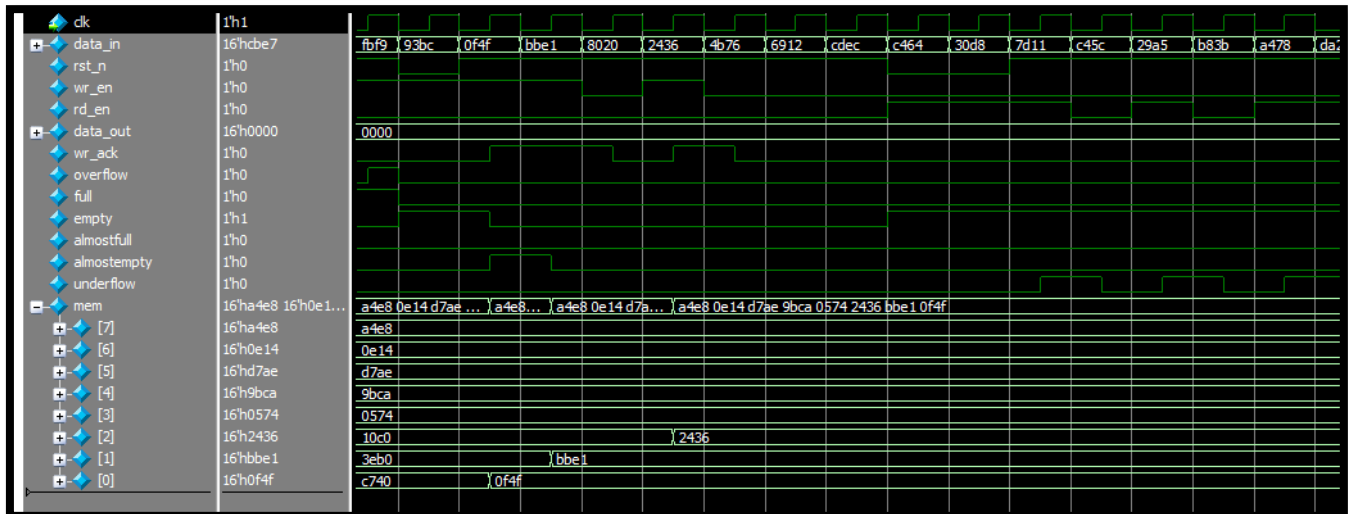
```
//Read only sequence=====
class FIFO_rd_only_seq extends uvm_sequence #(FIFO_seq_item);
`uvm_object_utils(FIFO_rd_only_seq)

FIFO_seq_item seq_item;

function new(string name = "FIFO_rd_only_seq");
    super.new(name);
endfunction

task body;
    seq_item = FIFO_seq_item::type_id::create("seq_item");

    repeat(100) begin
        start_item(seq_item);
        seq_item.RD_EN_ON_DIST = 70;
        seq_item.WR_EN_ON_DIST = 0;
        assert(seq_item.randomize());
        finish_item(seq_item);
    end
endtask
```



*Reading more than writing seq

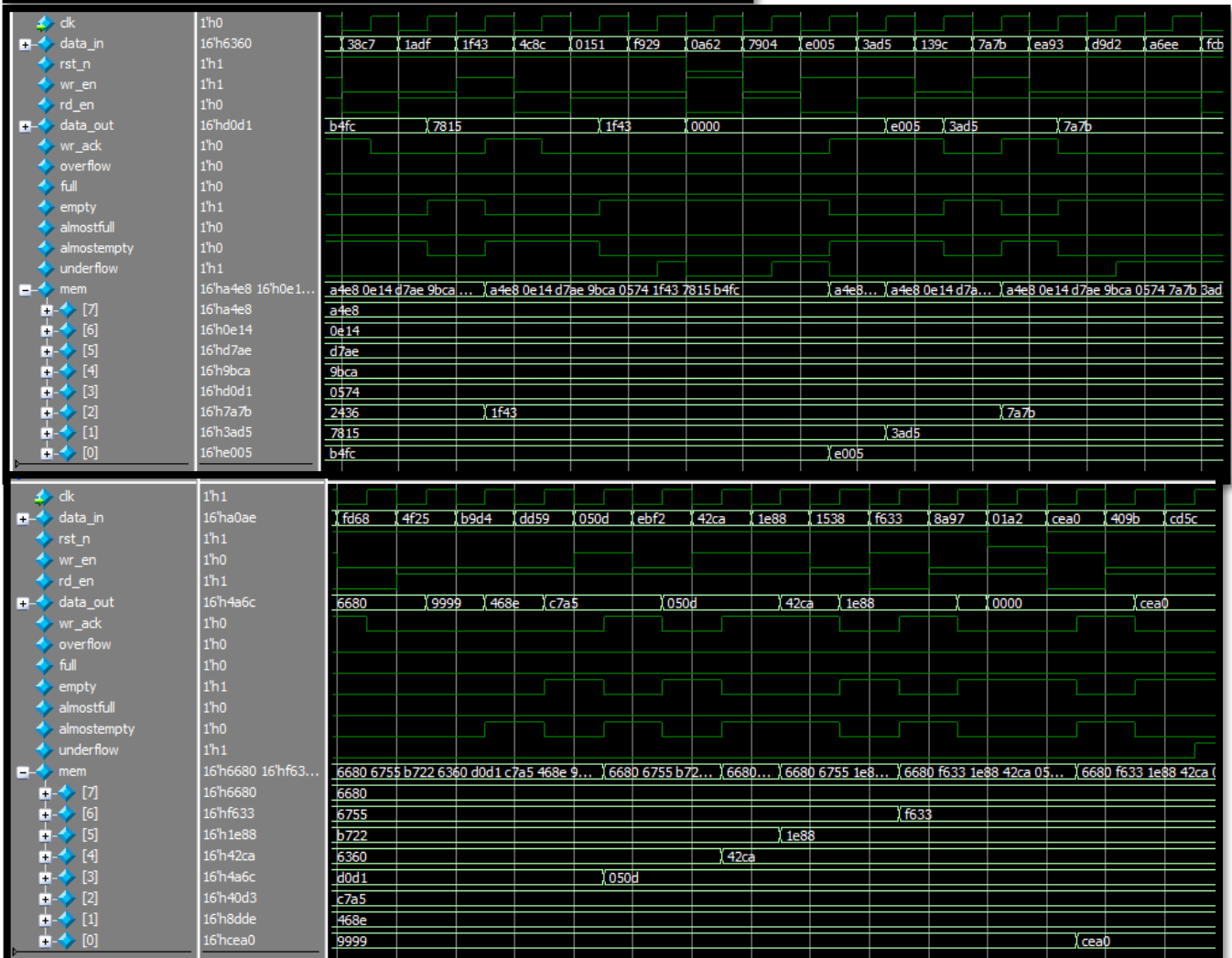
```
//Main sequence 1 (Read more probable than write)=====
class FIFO_main_seq_1 extends uvm_sequence #(FIFO_seq_item);
  `uvm_object_utils(FIFO_main_seq_1)

  FIFO_seq_item seq_item;

  function new(string name = "FIFO_main_seq_1");
    super.new(name);
  endfunction

  task body;
    seq_item = FIFO_seq_item::type_id::create("seq_item");

    repeat(100) begin
      start_item(seq_item);
      seq_item.RD_EN_ON_DIST = 70;
      seq_item.WR_EN_ON_DIST = 30;
      assert(seq_item.randomize());
      finish_item(seq_item);
    end
  endtask
endclass
```



***Writing more than reading seq**

```
//Main sequence 2 (Write more probable than read)=====
class FIFO_main_seq_2 extends uvm_sequence #(FIFO_seq_item);
    `uvm_object_utils(FIFO_main_seq_2)

    FIFO_seq_item seq_item;

    function new(string name = "FIFO_main_seq_2");
        super.new(name);
    endfunction

    task body;
        seq_item = FIFO_seq_item::type_id::create("seq_item");

        repeat(100) begin
            start_item(seq_item);
            seq_item.RD_EN_ON_DIST = 30;
            seq_item.WR_EN_ON_DIST = 70;
            assert(seq_item.randomize());
            finish_item(seq_item);
        end
    endtask
endclass
```

